

VISÃO COMPUTACIONAL

Seleção de Features, Classificadores Clássicos e Transfer Learning

Prof. Dr. José Jairo de S. e Silva

Aula 09

Aula 09

Seleção de Features, Classificadores Clássicos e Transfer Learning

OBJETIVOS DA AULA

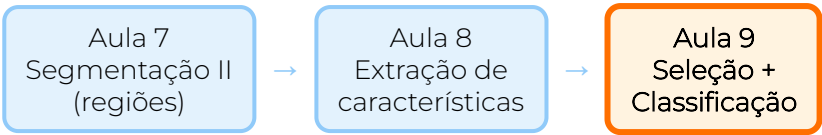
🎯 Objetivos de Aprendizagem

Ao final desta aula, o estudante será capaz de **selecionar** o subconjunto mais informativo de features extraídas, **treinar e comparar** classificadores clássicos (KNN, SVM, Random Forest, Regressão Logística) e **aproveitar redes pré-treinadas** via *transfer learning* para problemas com poucas amostras.

Conteúdos:

- Recap: vetor de features da Aula 8
- Por que selecionar? Maldição da dimensionalidade
- Famílias de seleção: **filter**, **wrapper**, **embedded**
- **VarianceThreshold**, **SelectKBest**, **RFE**, **feature_importances_**
- Pipeline de treino: **train_test_split**, **StandardScaler**
- KNN, Regressão Logística, SVM, Random Forest
- Validação cruzada e métricas (acurácia, F1, matriz, ROC)
- Comparação dos 4 classificadores no dataset de maçãs
- Introdução a redes neurais e Deep Learning
- **Transfer learning**: features de uma CNN pré-treinada

Onde a aula se encaixa



Conexões explícitas com aulas anteriores

Aula	O que entra na Aula 9
Aula 7	Máscaras de regiões (cada objeto isolado)
Aula 8	Vetor de features por região (cor, forma, textura) e a função <code>features_maca</code>
Aula 9 (atual)	Selecionar features úteis, treinar 4 classificadores, comparar com transfer learning

A passagem natural é: *vetor de features* (Aula 8) → *modelo de decisão* (Aula 9) → *fechamento do pipeline clássico de visão computacional*.

RECAP: DA AULA 8 PARA A AULA 9

O que a Aula 8 deixou pronto

Na Aula 8, construímos `features_maca(caminho)`, uma função que **recebe uma imagem** e devolve um dicionário com 11 descritores numéricos:

Família cor (HSV)

- `S_mean`, `V_mean`, `V_std`
- `frac_brown`: fração de pixels marrons
- `frac_dark`: fração de pixels escuros

Família forma (regionprops)

- `circularidade`
- `solidity`
- `eccentricity`

Família textura (GLCM + LBP)

- `contraste` (GLCM)
- `homogeneidade` (GLCM)
- `lbp_var`: variância do LBP

O que falta?

- **Selecionar** quais dessas 11 entram no modelo final.
- **Treinar** um classificador supervisionado.
- **Avaliar** com métricas adequadas.

A Aula 9 começa exatamente onde a Aula 8 parou: a tabela `X` de shape `(n_maças, 11)` com rótulos `y` em `{fresh, rotten}`.

A tabela X, y em maçãs

```
## frac_brown V_std circularidade solidity contraste homogeneidade classe
##      0.00 34.09          0.53      0.93      0.16          0.95 fresh
##      0.01 25.67          0.67      0.95      0.10          0.96 fresh
##      0.01 26.61          0.45      0.94      0.12          0.96 fresh
##      0.01 24.65          0.61      0.97      0.11          0.95 fresh

##
## ... (160 linhas no total; 11 features + 1 classe; mostradas 6 features e a classe para caber no slide)

## Classes: {'fresh': 80, 'rotten': 80}
```

Cada linha é uma maçã segmentada na Aula 8; cada coluna numérica é um descritor. A tabela completa tem **11 features** (mostramos 6 para caber); a coluna **classe** é o **rótulo supervisionado** (**fresh** ou **rotten**) que o classificador aprende a prever.

</> Código: construir a tabela X, y

```
import glob, pandas as pd
# features_maca: função da Aula 8 (devolve dict com 11 descritores)

linhas = []
for classe in ["fresh", "rotten"]:
    for caminho in sorted(glob.glob(f"img/apples/{classe}/*.jpg"))[:80]:
        feats = features_maca(caminho)          # dict com 11 chaves
        if feats is None:
            continue                            # segmentação falhou
        feats["classe"] = classe
        linhas.append(feats)

df = pd.DataFrame(linhas)                      # 160 linhas x 12 colunas
y = df["classe"].values                       # rótulos
X = df.drop(columns="classe").values          # matriz de features
print(X.shape, y.shape)                      # (160, 11) (160,)
```

Daqui em diante, **nada mais é específico de visão computacional**: **X** e **y** poderiam vir de qualquer fonte. Toda a infraestrutura do **scikit-learn** se aplica.

SELEÇÃO DE FEATURES

filter, wrapper, embedded

? Por que selecionar features?

Extrair 11 descritores é barato. Manter todos no classificador, porém, custa caro.

Três problemas

1. **Maldição da dimensionalidade**: em alta dimensão, a noção de "vizinho próximo" perde significado e o KNN degrada.
2. **Redundância**: duas features muito correlacionadas dobram o peso da mesma informação.
3. **Ruído**: uma feature que não separa as classes só adiciona variância, prejudica a generalização.

Em CV clássica, é comum ter mais features do que amostras úteis. Selecionar é parte do projeto, não uma etapa opcional.

O que selecionar resolve

- Modelo **mais simples**, com menos parâmetros a estimar.
- **Menor risco de overfitting**, especialmente com poucas amostras (160 maçãs, 11 features = relação 14:1).
- **Mais interpretável**: o time consegue dizer quais 3 ou 4 atributos guiam a decisão.
- Predição **mais rápida** em produção.

Três famílias de métodos

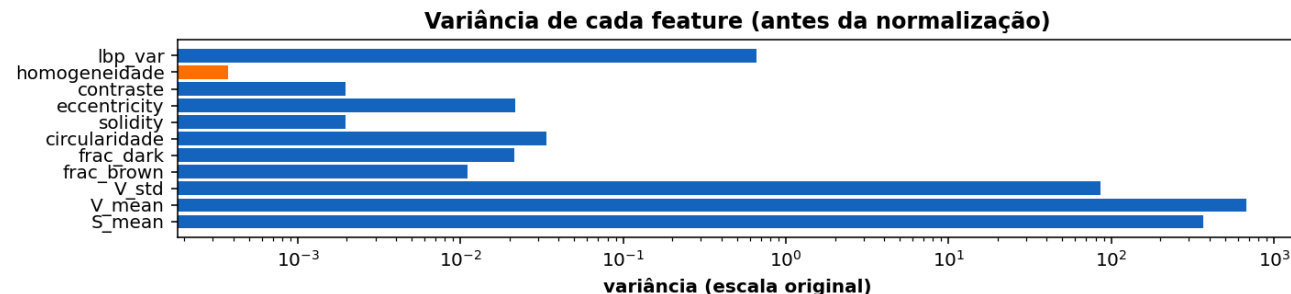
Família	Como decide	Custo	Exemplos <code>sklearn</code>
Filter	Estatística sobre cada feature isolada, antes do classificador	Baixo	<code>VarianceThreshold</code> , <code>SelectKBest</code> (<code>f_classif</code> , <code>mutual_info_classif</code> , <code>chi2</code>)
Wrapper	Testa subconjuntos rodando o classificador como caixa-preta	Alto	<code>RFE</code> , <code>RFECV</code> , <code>SequentialFeatureSelector</code>
Embedded	A seleção acontece durante o treino do classificador	Médio	<code>Lasso</code> (L1), <code>RandomForestClassifier.feature_importances_</code> , árvore boosting

Como escolher na prática

- **Poucos dados, muitas features:** comece por *filter* para descartar o óbvio.
- **Modelo barato:** *wrapper* fica viável (testa muitos subconjuntos).
- **Quer um modelo só:** *embedded* faz seleção e classificação na mesma chamada.

Esses três métodos **não competem**, eles se **combinam**. Um pipeline típico: filtra ruído com *filter*, refina com *wrapper* ou *embedded*.

▼ Filter 1: VarianceThreshold



VarianceThreshold descarta features cuja **variância** é menor que um limiar. Útil para remover descritores quase constantes (ex.: uma flag que vale 0 em quase toda amostra). Não usa o rótulo, é o filter mais simples.

Detalhes que importam

- **Aplique sempre depois do StandardScaler** (ou antes, sobre dados ainda em escalas próprias). Features em escalas muito diferentes (área em milhares vs circularidade em $[0,1]$) têm variâncias incomparáveis: a área "venceria" qualquer limiar mesmo sendo pouco informativa.
- **Limiar típico: 0.0** remove só constantes literais; **0.01** (após scaler) remove quase-constantes; **>0.1** começa a ser agressivo.

Quando usar e quando não

- **Use** quando o vetor de features pode ter colunas degeneradas (flags, contadores que ficam em zero).
- **Não use** como único método de seleção: ele não enxerga a relação com **y**. Uma feature pode variar bastante e não discriminar nada (ruído puro).
- **Combine** com **SelectKBest** ou **SelectFromModel** em seguida.

Em features manuais cuidadosamente extraídas (como as 11 da Aula 8), o **VarianceThreshold** raramente remove algo. Ele brilha em datasets brutos com centenas de colunas geradas automaticamente.

</> Código: VarianceThreshold

```
from sklearn.feature_selection import VarianceThreshold
from sklearn.preprocessing import StandardScaler

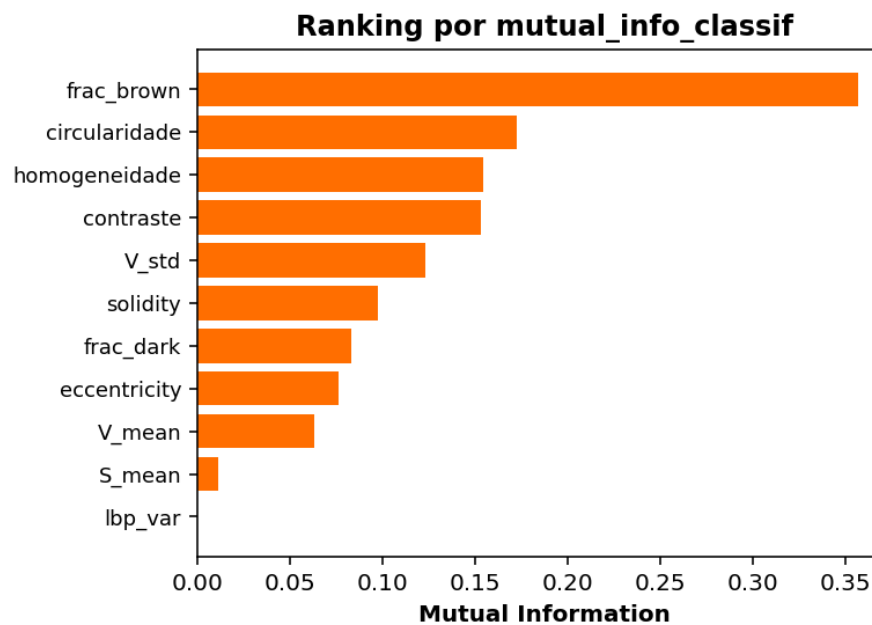
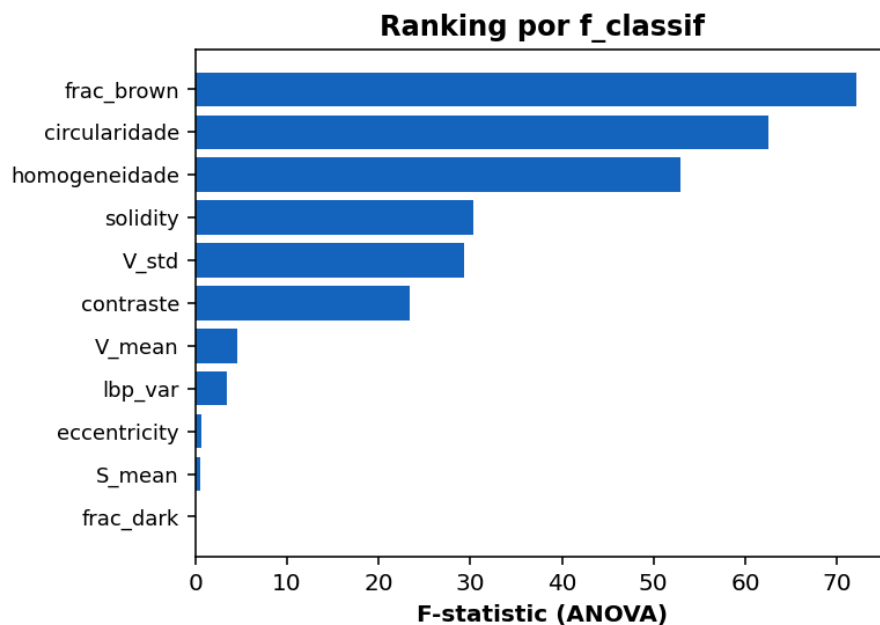
# IMPORTANTE: normalize ANTES de aplicar threshold de variância,
# senão features em escalas diferentes (ex.: area em milhares,
# circularidade em [0,1]) viram incomparáveis.
X_norm = StandardScaler().fit_transform(X)

vt = VarianceThreshold(threshold=0.01)
X_filtrado = vt.fit_transform(X_norm)

print(X.shape, "->", X_filtrado.shape)
print("Mantidas:", list(df.columns[:-1][vt.get_support()])))
```

Cuidado: **VarianceThreshold** não conhece o rótulo **y**. Ele só remove o que **não varia**, não o que **não importa**.

Filter 2: SelectKBest com `f_classif`



Cada feature recebe uma pontuação **em relação ao rótulo**. `f_classif` mede separação linear de médias por classe; `mutual_info_classif` captura também relações não lineares.

</> Código: SelectKBest

```
from sklearn.feature_selection import SelectKBest, f_classif, mutual_info_classif

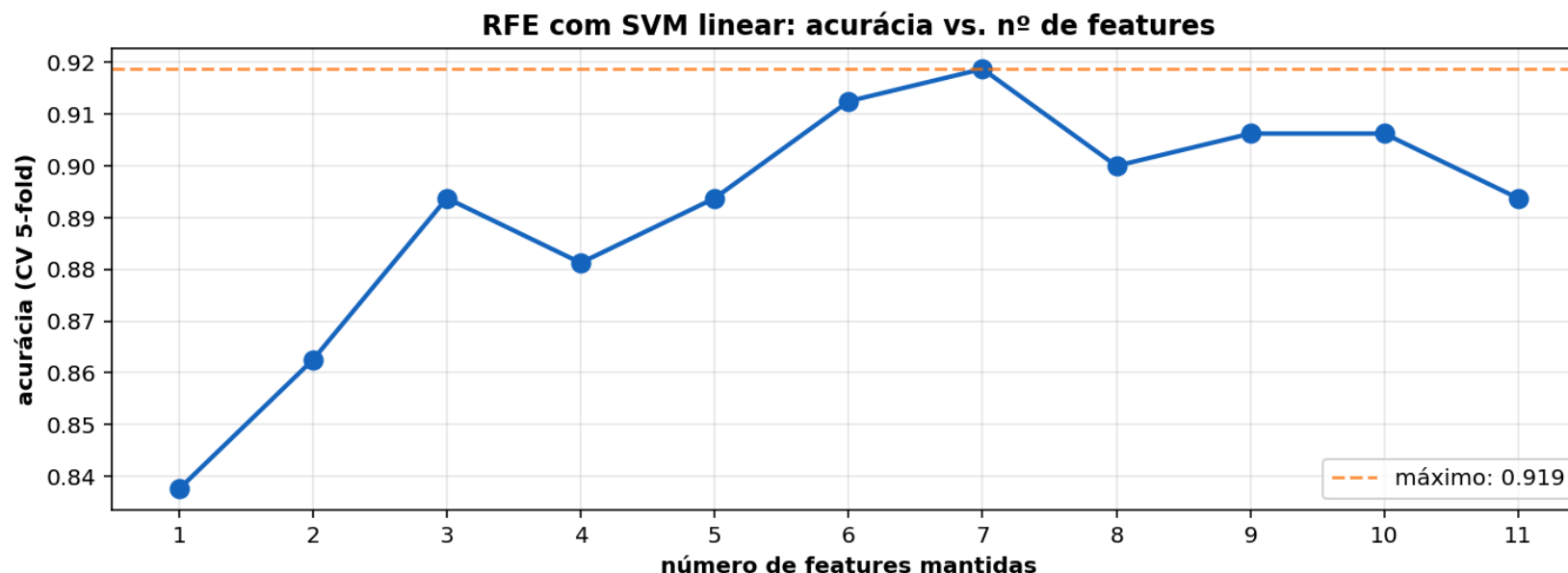
# Versão 1: F-statistic (ANOVA), assume médias separáveis linearmente
skb_f = SelectKBest(score_func=f_classif, k=5)
X_top5 = skb_f.fit_transform(X, y)

# Versão 2: mutual information, mais robusta a relações não lineares
skb_mi = SelectKBest(score_func=mutual_info_classif, k=5)
X_top5_mi = skb_mi.fit_transform(X, y)

# Quais features sobreviveram em cada ranking?
import numpy as np
nomes = np.array(df.columns.drop("classe"))
print("F-statistic top-5 :", nomes[skb_f.get_support()].tolist())
print("Mutual info top-5 :", nomes[skb_mi.get_support()].tolist())
```

Se as duas listas coincidem, há **consenso estatístico**. Se diferem muito, vale investigar: pode haver feature não-linear que o F-test não vê.

✂️ Wrapper: RFE (Recursive Feature Elimination)



RFE treina o classificador com **todas** as features, descarta a de menor coeficiente, retreina, descarta de novo, até restar o número desejado. A curva mostra o trade-off entre **número de features** e **acurácia**.

</> Código: RFECV

```
from sklearn.feature_selection import RFECV
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold

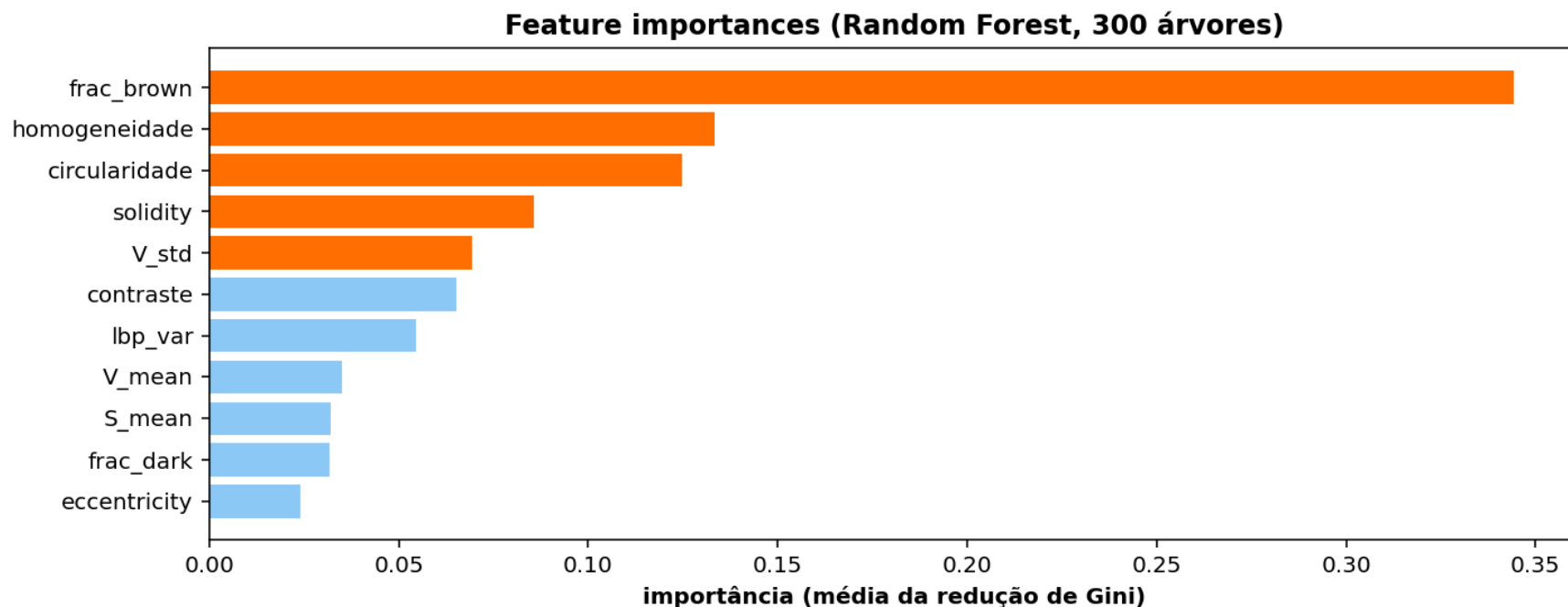
X_norm = StandardScaler().fit_transform(X)

# RFECV: faz validação cruzada para escolher k automaticamente.
# step=1 remove 1 feature por iteração; cv=5 usa 5-fold.
rfe = RFECV(
    estimator=SVC(kernel="linear", C=1.0),
    step=1,
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=0),
    scoring="accuracy",
)
rfe.fit(X_norm, y)

print(f"Melhor k = {rfe.n_features_}")
print("Features mantidas:", nomes[rfe.support_].tolist())
print(f"Ranking de eliminação: {rfe.ranking_}") # 1 = mantida
```

Wrapper é caro: para 11 features e 5-fold, são $11 \times 5 = 55$ ajustes. Em datasets grandes, prefira *filter* primeiro e *wrapper* só nas sobreviventes.

🌲 Embedded: feature_importances_ do Random Forest



Cada árvore mede quanto cada feature **reduz a impureza** de Gini ao ser usada em um split. A média sobre as 300 árvores produz um ranking estável. As 5 primeiras (laranja) concentram a maior parte do poder discriminativo.

</> Código: feature importance + SelectFromModel

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel
import pandas as pd

# 1. Treina o RF; ele aprende as importâncias durante o ajuste
rf = RandomForestClassifier(n_estimators=300, random_state=0)
rf.fit(X, y)

# 2. Inspecciona o ranking
ranking = pd.DataFrame({
    "feature": nomes,
    "importancia": rf.feature_importances_,
}).sort_values("importancia", ascending=False)
print(ranking.to_string(index=False))

# 3. SelectFromModel: corta automaticamente abaixo de um threshold
sel = SelectFromModel(rf, threshold="median", prefit=True)
X_sel = sel.transform(X)
print(X.shape, "->", X_sel.shape)
print("Mantidas:", [n for n, m in zip(nomes, sel.get_support()) if m])
```

Embedded é **gratuito** quando o classificador já é uma árvore: o ranking sai como **subproduto** do treino. Não há custo extra além do próprio fit.

✓ Resumo dos métodos de seleção

Método	Tipo	Usa y?	Considera interações?	Quando preferir
VarianceThreshold	Filter	Não	Não	Remover descritores quase constantes
SelectKBest(f_classif)	Filter	Sim	Não	Filtragem rápida, alvo categórico
SelectKBest(mutual_info)	Filter	Sim	Parcial	Relações não lineares feature → classe
RFE / RFECV	Wrapper	Sim	Sim	Modelo final é linear (SVM, LR)
feature_importances_ (RF)	Embedded	Sim	Sim	Árvores ou ensembles
Lasso (L1)	Embedded	Sim	Sim	Modelo final é linear, dados em alta dimensão

Ordem prática recomendada

1. **VarianceThreshold** para descartar lixo óbvio.
2. **SelectKBest** para uma redução rápida e barata.
3. **RFECV** ou **feature_importances_** na rodada final, antes do classificador definitivo.

Vale a pena selecionar? Resultados reais nas maçãs

Mesmos 4 classificadores, treinados sobre **3 conjuntos de features diferentes**, com 5-fold CV no dataset de 160 maçãs:

Classificador	11 features (todas)	5 features (SelectKBest)	3 features (RFECV)
KNN(k=5)	90,62% $\pm$ 1,98	88,75% $\pm$ 4,68	88,12% $\pm$ 3,64
Regressão Logística	90,62% $\pm$ 2,80	91,25% $\pm$ 4,59	90,00% $\pm$ 3,64
SVM (rbf)	93,12% $\pm$ 2,34	88,75% $\pm$ 3,19	89,38% $\pm$ 2,50
Random Forest	92,50% $\pm$ 3,19	90,62% $\pm$ 3,95	90,00% $\pm$ 3,06

O que esses números dizem

- **A Regressão Logística MELHORA** quando seleciona (91,25% > 90,62%). Modelo linear se beneficia de remover ruído.
- **KNN, SVM(rbf) e Random Forest perdem 1-3 pontos** com menos features. Eles **modelam interações** que se perdem quando colunas são removidas.
- Os ganhos/perdas estão dentro do **desvio padrão do CV ($\pm$ 2-4%)**, então a diferença não é estatisticamente decisiva neste dataset pequeno.

Regra: se o dataset é pequeno e o modelo é não-linear (SVM-rbf, RF), começar com as features completas costuma ganhar. Selecionar entra quando se busca **interpretabilidade**, **menos overfitting** ou **modelos lineares**.

Quando a seleção compensa de fato

- **Datasets grandes** com muitas features (>50): redução de overfitting e tempo de treino é real.
- **Modelos lineares** (LogReg, SVM linear): seleção remove ruído que atrapalha.
- **Modelos baseados em árvore** (RF, GBoost): geralmente **não precisam**, eles próprios ignoram features irrelevantes.
- **Interpretabilidade exigida:** 3 features para explicar > 11 features para defender.

CLASSIFICADORES CLÁSSICOS

KNN, Regressão Logística, SVM, Random Forest

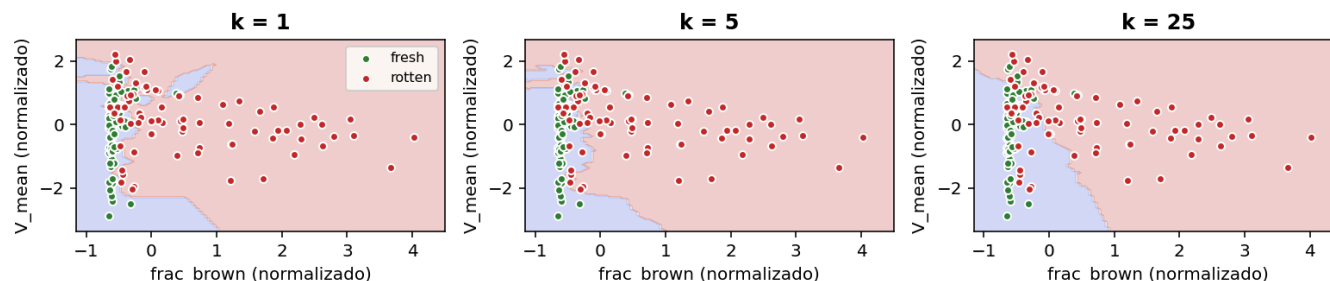
Os 4 classificadores em uma tabela

Classificador	Ideia central	Hiperparâmetros	Sensível à escala?
KNN	Voto dos k vizinhos mais próximos no espaço de features	n_neighbors , metric	Sim
Regressão Logística	Fronteira linear + sigmoide para gerar probabilidade	C (regularização)	Sim
SVM	Hiperplano de margem máxima (com truque de kernel)	C , kernel , gamma	Sim
Random Forest	Conjunto de árvores de decisão, voto majoritário	n_estimators , max_depth	Não

Princípios em comum

- Todos têm a mesma **interface scikit-learn**: **.fit(X, y)**, **.predict(X)**, **.predict_proba(X)**.
- Todos suportam **validação cruzada** com **cross_val_score**.
- Os três primeiros exigem **StandardScaler**; o Random Forest é indiferente à escala porque árvores só comparam valores em cada feature isolada.

KNN: voto dos vizinhos próximos



$k=1$ cria fronteiras **muito recortadas** (overfitting). $k=25$ produz fronteiras **suaves** (underfitting). Existe um valor intermediário que generaliza melhor.

Como o KNN funciona, em 4 passos

1. **Treino**: o KNN apenas **armazena** o X_{treino} e y_{treino} . Não há ajuste de parâmetros, não há "modelo" interno.
2. **Predição**: para cada nova amostra, calcula a **distância** (euclidiana, por padrão) a todas as amostras de treino.
3. Escolhe os **k mais próximos** ($n_{\text{neighbors}}$).
4. **Vota** pela classe majoritária ($\text{weights}=\text{"uniform"}$) ou pela classe mais próxima ponderada ($\text{weights}=\text{"distance"}$).

Hiperparâmetros que importam

- **$n_{\text{neighbors}}$** : o trade-off central. Pequeno = decora ruído; grande = ignora estrutura local.
- **$\text{weights}=\text{"distance"}$** : vizinhos próximos pesam mais que os distantes. Recomendado quase sempre.
- **$\text{metric}:\text{"minkowski"}$** (default, generaliza euclidiana). **"cosine"** para features de texto/embeddings.
- **StandardScaler é OBRIGATÓRIO**: distância euclidiana é dominada pela feature de maior escala se não normalizar.

</> Código: KNN com pipeline

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# StandardScaler é OBRIGATÓRIO no KNN.
# Sem ele, uma feature em escala "milhares" domina a distância
# euclidiana e features em [0,1] passam a contar quase nada.
knn = Pipeline([
    ("sc", StandardScaler()),
    ("clf", KNeighborsClassifier(n_neighbors=5, weights="distance")),
])

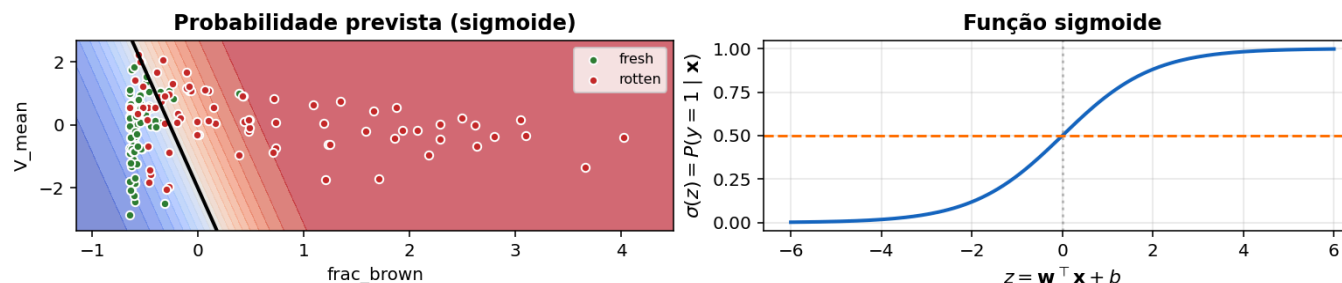
knn.fit(X_treino, y_treino)
y_pred = knn.predict(X_teste)
```

Decisões de projeto

- `n_neighbors`: pequeno = mais flexível, grande = mais suave.
- `weights="distance"`: vizinhos próximos pesam mais (recomendado).
- `metric="minkowski"` (default) → distância euclidiana.

KNN não tem fase de treino real: apenas memoriza `X_treino`. O custo está no `.predict`, que busca os `k` vizinhos mais próximos a cada nova amostra.

Regressão Logística: fronteira linear + sigmoide



A logística aprende uma combinação linear $w \cdot x + b$, passa pela sigmoide e devolve uma **probabilidade**. A fronteira de decisão é a linha onde $P = 0,5$.

Como funciona

- Modelo: $P(y = 1 | x) = \sigma(w^T x + b)$, com $\sigma(z) = \frac{1}{1+e^{-z}}$.
- Treino: maximiza a **verossimilhança** (equivalente a minimizar log-loss) usando gradiente.
- A sigmoide mapeia qualquer número real para **(0, 1)**, perfeito para probabilidade.
- **Fronteira linear no espaço de features**: o conjunto $\{x : w^T x + b = 0\}$ é um hiperplano.
- Para multiclasse: extensão **softmax** (várias saídas que somam 1).

Hiperparâmetros que importam

- **C** (inverso da regularização): **C** baixo = mais regularização = coeficientes menores. Padrão: **1.0**.
- **penalty**: "**l2**" (padrão, encolhe todos os coeficientes), "**l1**" (zera os irrelevantes, faz seleção embutida), "**elasticnet**" (combinação).
- **solver**: "**liblinear**" para L1 com binário; "**saga**" para L1 multiclasse; "**lbfgs**" (padrão) para L2.
- **max_iter**: aumente para **1000** ou **2000** se sklearn reclamar de não-convergência.
- **StandardScaler ajuda muito**: sem ele, a regularização L1/L2 afeta features em escalas diferentes de forma desigual.

Os coeficientes `lr.coef_[0]` são interpretáveis: **positivo** empurra para classe 1 (rotten), **negativo** para classe 0 (fresh). É a feature interpretability mais simples e barata.

</> Código: Regressão Logística

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

lr = Pipeline([
    ("sc", StandardScaler()),
    ("clf", LogisticRegression(C=1.0, penalty="l2", max_iter=1000)),
])
lr.fit(X_treino, y_treino)

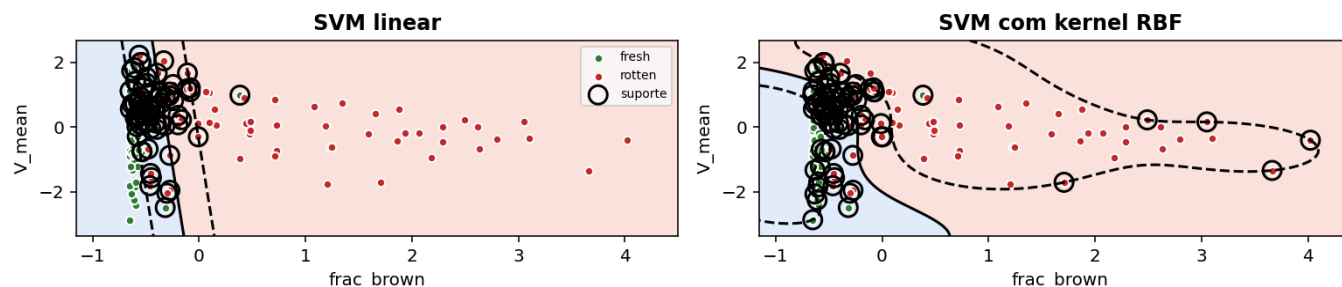
probas = lr.predict_proba(X_teste)[: , 1] # P(rotten) para cada maçã
y_pred = (probas > 0.5).astype(int)

# Coeficientes: quanto cada feature pesa na decisão linear
import pandas as pd
coef = pd.Series(lr.named_steps["clf"].coef_[0], index=nomes)
print(coef.sort_values(ascending=False))
```

Decisões de projeto

- **C** pequeno = mais regularização (coeficientes mais próximos de zero).
- **penalty="l1"** força esparsidade: alguns coeficientes viram **exatamente zero** (seleção de features embedded).
- Saída sempre **interpretável**: o sinal de cada coeficiente indica para qual classe a feature empurra.

SVM: hiperplano de margem máxima



SVM escolhe o hiperplano com **maior margem** até os pontos mais próximos (vetores de suporte). Com `kernel="rbf"`, a margem é construída em um espaço transformado, gerando fronteira **não linear** na visualização original.

Como funciona

- **Margem máxima**: entre todas as fronteiras que separam as classes, o SVM escolhe aquela cuja distância aos pontos mais próximos é **a maior possível**. A intuição: quanto maior a margem, mais robusto a ruído.
- **Vetores de suporte** (círculos pretos na figura): só os pontos **na borda** ou **dentro** da margem participam da decisão. Removê-los muda a fronteira; os outros pontos não.
- **Kernel trick (RBF)**: em vez de calcular coordenadas em um espaço de dimensão alta, o SVM calcula apenas **distâncias** nesse espaço via uma função kernel. Permite fronteiras não-lineares mantendo o problema tratável.

Hiperparâmetros críticos

- **C** (regularização): **C** alto = pune mais erros no treino = margem mais estreita = risco de overfit. **C** baixo = margem generosa = mais robusto.
- **gamma** (só no RBF): controla a "largura" do kernel. **gamma** alto = fronteiras muito curvas (overfit); **gamma="scale"** é um bom default.
- **probability=True**: habilita **predict_proba** (faz Platt scaling internamente, custa tempo extra no fit).
- **StandardScaler é obrigatório**, igual ao KNN: o SVM usa distâncias.

SVM funciona bem em datasets **médios** (centenas a milhares de amostras). Para milhões, troque por **LinearSVC** ou **SGDClassifier(loss="hinge")**. Para muitas classes, considera **OneVsRest**.

</> Código: SVM

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

svm = Pipeline([
    ("sc", StandardScaler()),
    ("clf", SVC(kernel="rbf", C=1.0, gamma="scale", probability=True)),
])
svm.fit(X_treino, y_treino)
y_pred = svm.predict(X_teste)

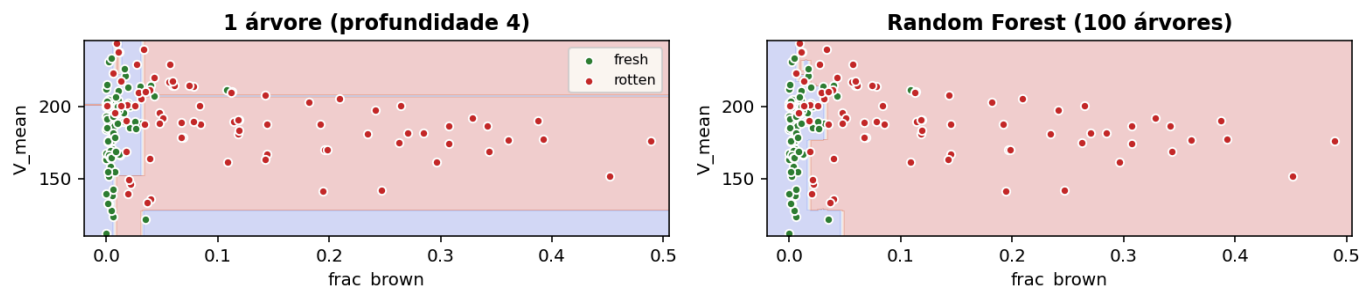
# probability=True habilita predict_proba (custa um Platt scaling interno)
prob_rotten = svm.predict_proba(X_teste)[:, 1]
```

Decisões de projeto

- `kernel="linear"`: rápido, interpretável, suficiente quando classes são separáveis por hiperplano.
- `kernel="rbf"`: padrão para problemas não lineares, exige ajustar `gamma`.
- `C`: penaliza erros de classificação. `C` alto = margem mais estreita, menos erro no treino, mais risco de overfitting.

O custo de treino do SVM cresce com o número de amostras ($O(n^2)$ ou pior). Para datasets gigantes, prefira `LinearSVC` ou `SGDClassifier(loss="hinge")`.

🌲 Random Forest: ensemble de árvores



Uma árvore isolada cria fronteiras **retangulares** e tende a overfittar. O Random Forest treina muitas árvores em **subconjuntos aleatórios** (bagging) e produz uma fronteira mais suave e robusta.

Como o Random Forest funciona

1. **Bagging**: para cada uma das **n_estimators** árvores, o RF sorteia uma **amostra com reposição** dos dados de treino (bootstrap).
2. **Feature randomization**: a cada split de uma árvore, considera apenas **sqrt(n_features)** colunas escolhidas ao acaso (não todas).
3. **Treino independente** de cada árvore (não há comunicação entre elas).
4. **Predição**: cada árvore vota; a classe majoritária vence. Para regressão, faz a média.
5. Os dois ingredientes (bagging + feature randomization) garantem que as árvores sejam **descorrelacionadas**, e a média de modelos descorrelacionados reduz variância sem aumentar viés.

Hiperparâmetros que importam

- **n_estimators**: mais árvores = mais estável até saturar. **100 a 500** cobre a maioria dos casos. Custo cresce linearmente.
- **max_depth**: árvores muito profundas overfitting; muito rasas underfitting. **None** (default) deixa crescer; vale tentar **max_depth=10** para datasets pequenos.
- **min_samples_leaf**: número mínimo de amostras por folha. Aumentar regulariza.
- **max_features**: "**sqrt**" é o padrão; reduzir aumenta diversidade.
- **StandardScaler NÃO é necessário**: árvores fazem comparações dentro de cada coluna isolada.
- **Grátis no fit**: **feature_importances_** (importância média via Gini).

Random Forest é o **default sensato** para tabelas numéricas até $\sim 10^5$ linhas. Para datasets maiores ou com fortes interações entre features, **Gradient Boosting** (XGBoost, LightGBM) costuma vencer.

</> Código: Random Forest

```
from sklearn.ensemble import RandomForestClassifier

# Random Forest NÃO precisa de StandardScaler:
# árvores só fazem comparações dentro de cada feature isolada.
rf = RandomForestClassifier(
    n_estimators=300,          # quantas árvores
    max_depth=None,           # None = cresce até purificar (pode overfittar)
    min_samples_leaf=2,       # regularização: folha precisa ter pelo menos 2
    n_jobs=-1,                # usa todos os cores
    random_state=0,
)
rf.fit(X_treino, y_treino)
y_pred = rf.predict(X_teste)

# Bônus: importância de cada feature já vem treinada
print(sorted(zip(nomes, rf.feature_importances_),
              key=lambda x: -x[1])[0:5])
```

O Random Forest é o **caminho mais curto** para uma baseline forte: poucos hiperparâmetros realmente importam, lida bem com features em escalas diferentes e devolve ranking grátis.

PIPELINE DE TREINO E AVALIAÇÃO

✂ Divisão treino / teste

```
from sklearn.model_selection import train_test_split
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y,
    test_size=0.25,          # 75% treino, 25% teste
    stratify=y,              # mantém a proporção de classes em ambos os lados
    random_state=42,         # reprodutibilidade
)
```

Por que stratify=y?

Sem estratificação, o split aleatório pode deixar o conjunto de teste com **80% de uma classe e 20% da outra**. A acurácia fica enganosamente alta (basta o classificador chutar a classe majoritária).

Por que StandardScaler dentro de um Pipeline?

A normalização **deve ser aprendida no treino** (média e desvio das 120 maçãs de treino), e **aplicada ao teste** com esses mesmos parâmetros. Calcular média/desvio sobre **X** inteiro vaza informação do teste para o treino, inflando artificialmente o desempenho.

Pipeline garante isso: quando você chama `pipe.fit(X_tr, y_tr)`, o **StandardScaler** aprende **só com X_tr**; em `pipe.predict(X_te)`, ele transforma **X_te** com os parâmetros do treino.

↻ Validação cruzada

Com **160 amostras**, uma única divisão treino/teste é frágil: o resultado depende muito de quais maçãs caíram em cada lado. A solução é repetir a divisão.

K-Fold (StratifiedKFold)

1. Divide **X** em **k partes iguais** (mantendo a proporção de classes).
2. Treina em **k-1** partes, avalia na restante.
3. Repete **k** vezes, cada parte vira teste uma vez.
4. Reporta **média e desvio** das **k** métricas.

```
from sklearn.model_selection import cross_val_score\nscores = cross_val_score(\n    pipe, X, y,\n    cv=5, scoring="accuracy")\nprint(f"{scores.mean():.3f} ± {scores.std():"
```

○ que aparece no relatório

- **Média**: estimativa do desempenho real.
- **Desvio padrão**: medida de **estabilidade**. Desvio alto significa que o classificador é sensível à divisão dos dados.
- Reportar só a média esconde instabilidade. Sempre apresente os dois.

Para dataset pequeno e balanceado, **cv=5** é o padrão. Para muito pequeno ($n < 50$), considere **LeaveOneOut**.

☰ Métricas: além da acurácia

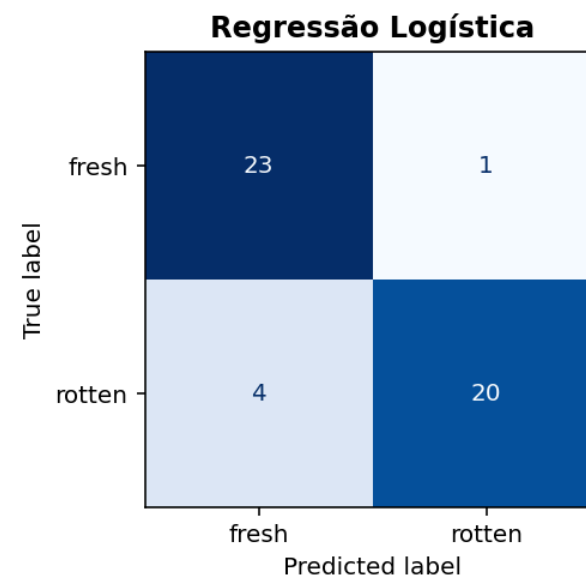
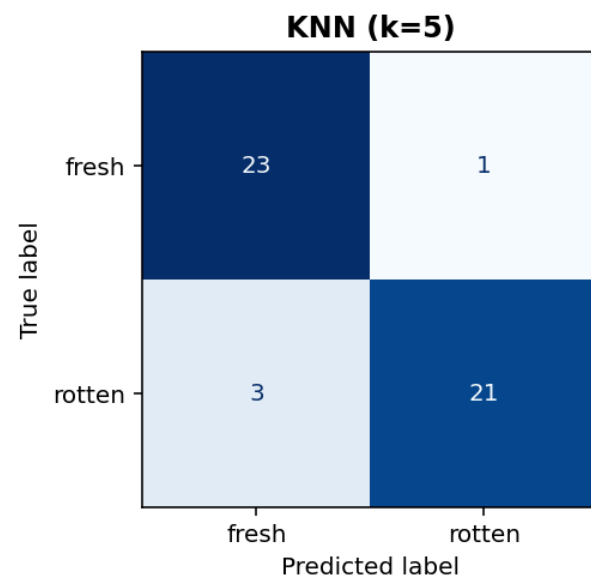
A acurácia funciona quando as classes são **balanceadas** e os erros têm **custo simétrico**. Quando não, precisamos de mais.

Métrica	Pergunta que responde	Fórmula
Acurácia	Que fração de previsões está correta?	$(TP+TN)/total$
Precisão	Dos que prevejo "rotten", quantos são mesmo?	$TP/(TP+FP)$
Recall	Das rotten reais, quantas eu peguei?	$TP/(TP+FN)$
F1	Equilíbrio entre precisão e recall	$2 \cdot P \cdot R / (P+R)$
ROC AUC	Capacidade de ranquear rotten vs fresh independente do limiar	área sob ROC

Cenário industrial: triagem de maçãs para venda

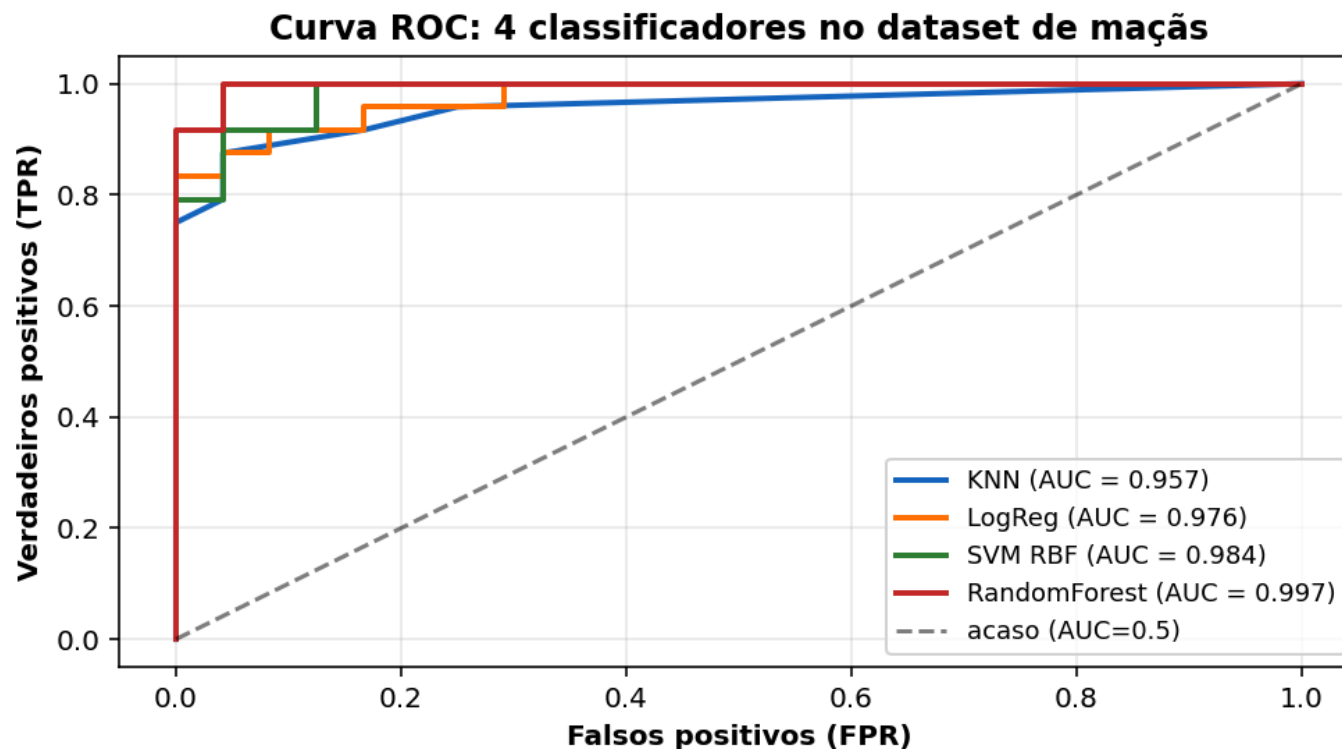
- *Custo de deixar passar uma rotten* (FN): cliente devolve a caixa inteira.
- *Custo de descartar uma fresh* (FP): perda pequena de uma maçã boa.
- Aqui, queremos **recall alto** para a classe **rotten** (não deixar passar), mesmo aceitando recall menor para **fresh**.

Matriz de confusão: leitura visual



Cada **diagonal** é acerto, cada **fora da diagonal** é tipo de erro. Identificar **onde** o classificador erra (sempre fresh, sempre rotten, simétrico?) costuma ser mais útil do que a acurácia final.

Curva ROC e AUC



A **AUC** independe do limiar de decisão (0,5 não é sagrado). Quanto mais perto de 1, melhor o classificador **ranqueia** rotten acima de fresh.

</> Código: relatório completo de métricas

```
from sklearn.metrics import (
    accuracy_score, classification_report,
    confusion_matrix, roc_auc_score,
    ConfusionMatrixDisplay, RocCurveDisplay,
)

# Após pipe.fit(X_tr, y_tr) e pipe.predict(X_te):
y_pred = pipe.predict(X_te)
y_prob = pipe.predict_proba(X_te)[:, 1]  # P(rotten)

print(f"Acurácia: {accuracy_score(y_te, y_pred):.3f}")
print(f"ROC AUC : {roc_auc_score(y_te, y_prob):.3f}\n")
print(classification_report(y_te, y_pred,
                           target_names=["fresh", "rotten"]))

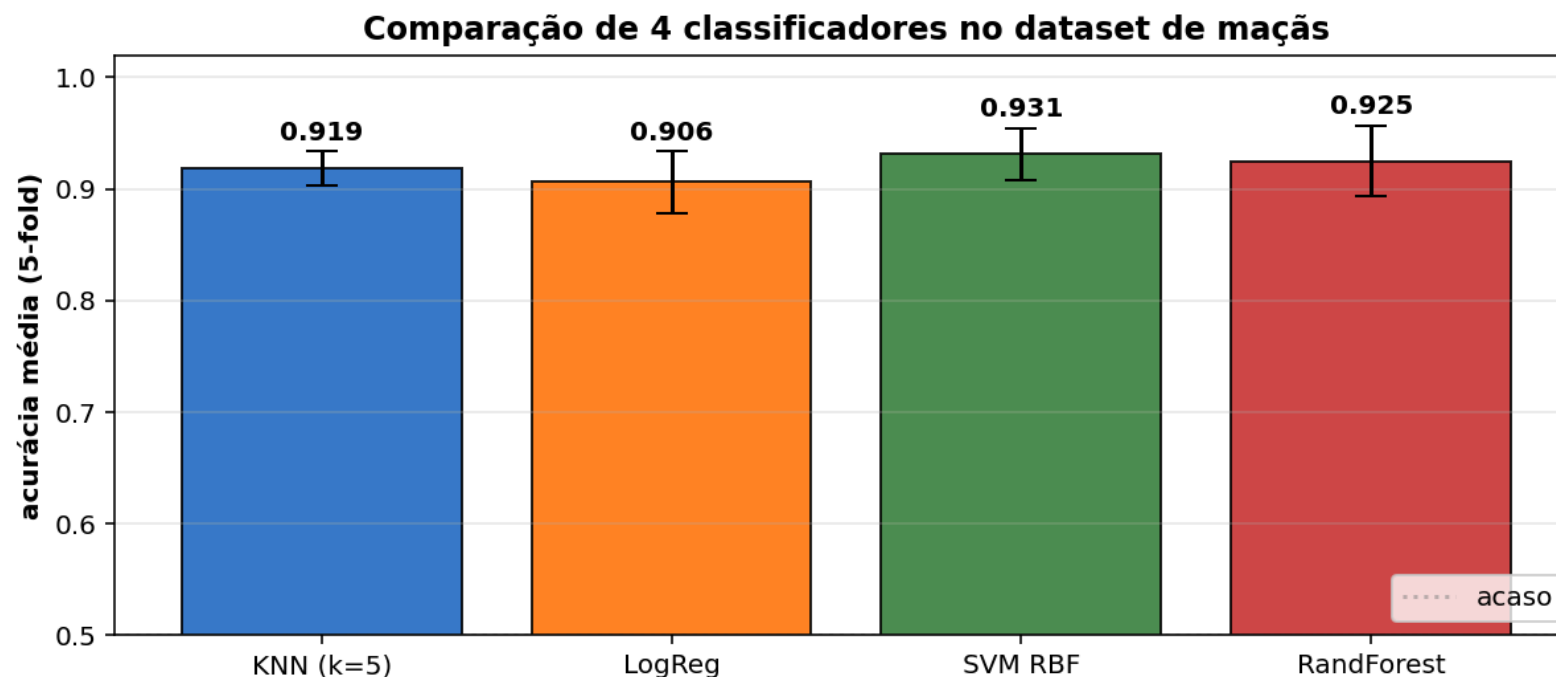
# Visualização gráfica
import matplotlib.pyplot as plt
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ConfusionMatrixDisplay.from_predictions(
    y_te, y_pred, display_labels=["fresh", "rotten"], ax=ax[0], cmap="Blues")
RocCurveDisplay.from_predictions(y_te, y_prob, ax=ax[1])
plt.tight_layout(); plt.show()
```

Reporte sempre **mais de uma métrica**: acurácia + matriz + classification report. Cada métrica revela um lado distinto do desempenho.

COMPARAÇÃO DOS 4 CLASSIFICADORES

no dataset de maçãs

■ Acurácia média (5-fold CV) com erro padrão



Os 4 classificadores ficam em **patamar parecido** porque as 11 features clássicas já carregam quase toda a informação **separável** do problema. O ganho marginal entre eles é menor que a variância da própria validação cruzada.

</> Código: comparar 4 classificadores

```
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
import pandas as pd

clfs = {
    "KNN": Pipeline([("sc", StandardScaler()),
                     ("clf", KNeighborsClassifier(n_neighbors=5,
                                                  weights="distance"))]),
    "LogReg": Pipeline([("sc", StandardScaler()),
                        ("clf", LogisticRegression(max_iter=1000))]),
    "SVM": Pipeline([("sc", StandardScaler()),
                     ("clf", SVC(kernel="rbf", probability=True))]),
    "RandFor": RandomForestClassifier(n_estimators=300, random_state=0),
}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

linhas = []
for nome, clf in clfs.items():
    acc = cross_val_score(clf, X, y, cv=cv, scoring="accuracy")
    f1 = cross_val_score(clf, X, y, cv=cv, scoring="f1")
    linhas.append({"clf": nome,
                  "acc_mean": acc.mean(), "acc_std": acc.std(),
                  "f1_mean": f1.mean(), "f1_std": f1.std()})
print(pd.DataFrame(linhas).round(3).to_string(index=False))
```

A "melhor escolha" depende do contexto: se prioriza interpretabilidade, **LogReg**; se prioriza ganho rápido sem ajustar, **RF**; se já há sinal não linear claro, **SVM RBF**; se o dataset é pequeno e bem comportado, **KNN** pode liderar.

INTRODUÇÃO A DEEP LEARNING

por que mudar de paradigma?

⚠ Limites da abordagem clássica

O que fizemos até aqui:

imagem → segmentação → features manuais → classificador
(11 descritores escolhidos pelo professor)

Funciona muito bem quando:

- O domínio é conhecido (a maçã tem cor, forma e textura típicas).
- O número de classes é pequeno (2 a 10).
- O dataset é pequeno (dezenas a poucas centenas de imagens).

Onde a abordagem clássica falha

Problema	Por quê
Reconhecer mil classes (cachorro, gato, avião, ...)	Inviável projetar features manuais para cada
Imagens com fundo complexo, sem segmentação confiável	Pipeline trava no primeiro passo
Variações de pose, iluminação, oclusão	Cada variação exige nova feature
Tarefa de detecção (onde está o objeto?)	Features de região exigem caixas, círculo vicioso

▮ Saída: deixar a rede aprender quais features extrair, em vez de projetá-las à mão.

O neurônio artificial

Um neurônio recebe um vetor $\mathbf{x}$, aplica uma combinação linear e uma função de ativação:

$$y = \phi\left(\sum_{i=1}^n w_i x_i + b\right) = \phi(\mathbf{w}^\top \mathbf{x} + b)$$

- $\mathbf{w}$ são os pesos aprendidos durante o treino.
- b é o bias (deslocamento).
- ϕ é a função de ativação (ex.: ReLU, sigmoide, tanh).

Coincidência intencional

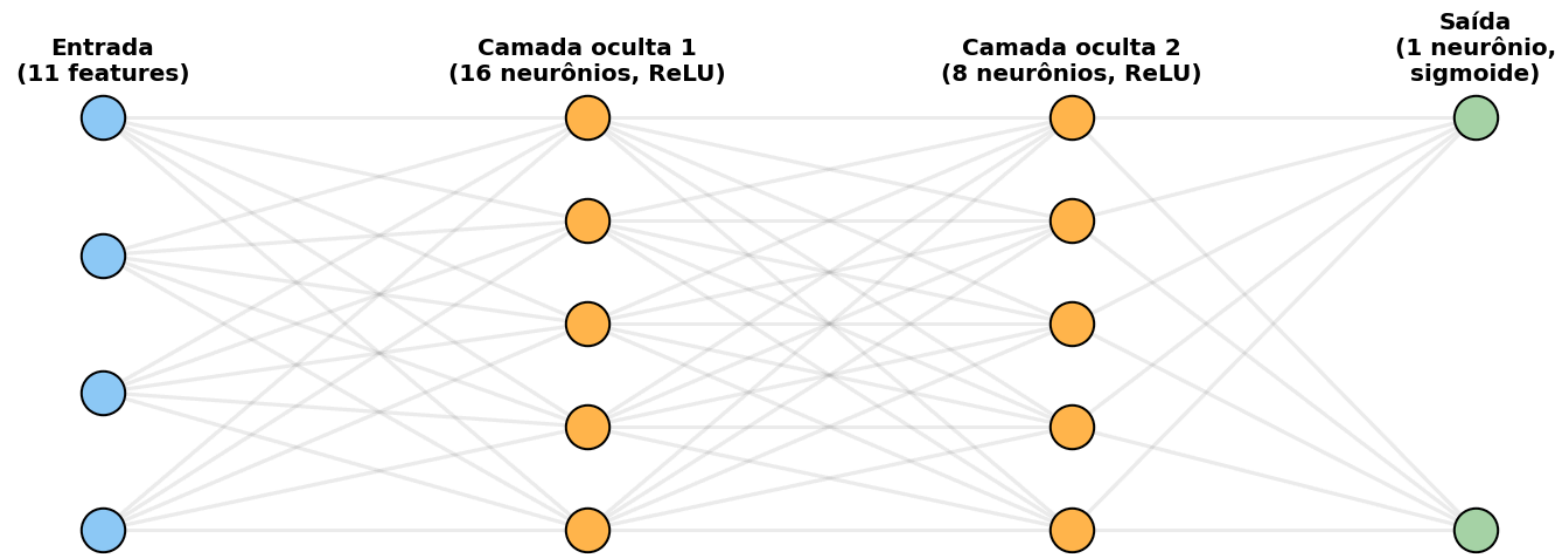
A Regressão Logística que treinamos no slide anterior é exatamente um neurônio com ativação sigmoide. A diferença é que ela tem um único neurônio decidindo a classe.

Uma rede neural empilha muitos neurônios em camadas, cada um aprendendo uma transformação útil da entrada.

Por que importa para nós?

- O classificador linear já era um caso particular de rede.
- Adicionar camadas permite aprender funções não lineares complexas.
- Para imagens, camadas convolucionais (CNN) substituem totalmente a engenharia manual de features, mas exigem muito mais dados e poder computacional.

Do neurônio à rede: MLP



Um **MLP** (Multi-Layer Perceptron) empilha camadas densas. Cada neurônio da camada **k+1** recebe **todas as saídas** da camada **k**. Toda essa rede é treinada via *backpropagation* e *gradient descent*.

</> Código: MLP simples em scikit-learn

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

mlp = Pipeline([
    ("sc", StandardScaler()),          # ESSENCIAL para redes
    ("clf", MLPClassifier(
        hidden_layer_sizes=(16, 8),    # 2 camadas ocultas
        activation="relu", max_iter=2000,
        early_stopping=True, random_state=0,
    )),
])
mlp.fit(X_treino, y_treino)
print("Acurácia:", mlp.score(X_teste, y_teste))
```

Limites do MLP em imagens

- Recebe um **vetor**. Para usar uma imagem inteira, é preciso **achatar** (**flatten**), o que destrói a estrutura espacial.
 - $224 \times 224 \times 3 = 150$ mil entradas, exige redes enormes e muitos dados. A solução para imagens é a **rede convolucional (CNN)**, que está fora do escopo desta disciplina mas é o caminho natural se quiserem aprofundar.
- O MLP é útil aqui só para mostrar a **transição conceitual**: a rede generaliza a Regressão Logística.

TRANSFER LEARNING

usar uma rede pré-treinada como extrator de features

💡 A ideia central

Treinar uma CNN do zero exige **milhões de imagens rotuladas** e GPUs potentes. Pouquíssimos times têm esses recursos. *Transfer learning* resolve isso explorando dois fatos empíricos:

Fato 1: as camadas iniciais aprendem features genéricas

- Bordas, cantos, manchas de cor, padrões repetitivos.
- Servem para **qualquer** tarefa visual: gatos, carros, satélites, maçãs.

Fato 2: existem redes prontas

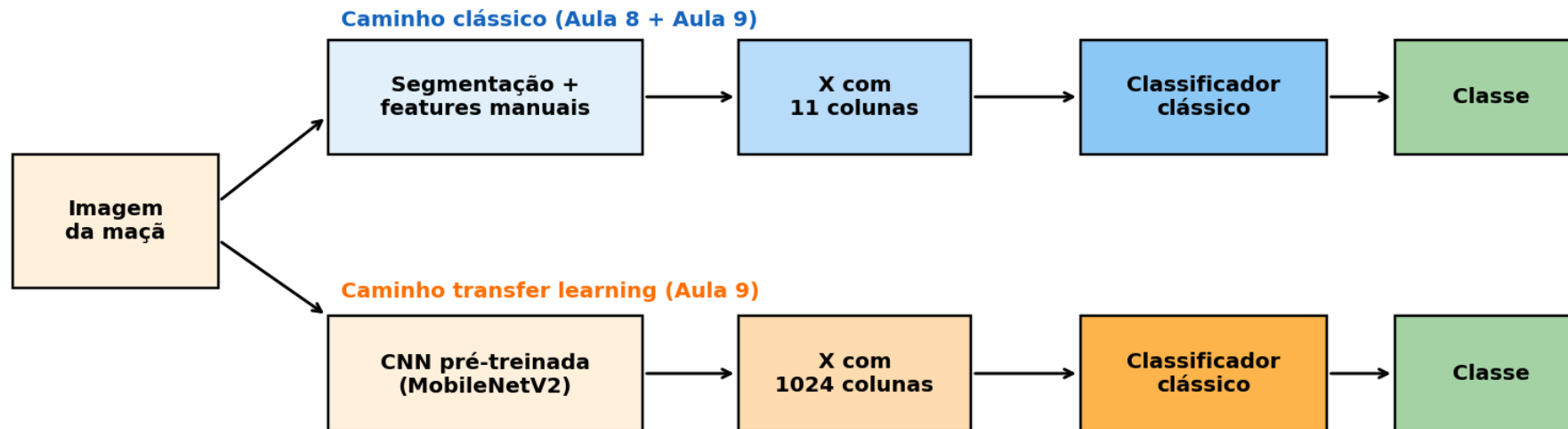
- Treinadas em **ImageNet** (1,2 milhão de fotos, mil classes).
- Disponíveis em **keras.applications**: MobileNet, ResNet, EfficientNet, VGG.
- Pesos baixados automaticamente.

Como aproveitamos

1. Carregar a CNN pré-treinada **sem a última camada** (sem o "classificador ImageNet").
2. Passar cada maçã pela rede e ler a **saída da camada anterior**: um vetor de 1024 dimensões (no caso da MobileNetV2).
3. Tratar esse vetor como **nosso novo X**, e treinar um classificador clássico em cima.
4. Comparar com o **X** de 11 features manuais.

A rede atua como **extrator de features**. O classificador final pode ser **a mesma Regressão Logística** da seção anterior.

Pipeline visual: dois caminhos para a maçã



A **única diferença** entre os dois caminhos é a fonte do vetor de features. Tudo depois (classificador, validação cruzada, métricas) permanece **igual**.

</> Código: extrair features com MobileNetV2

```
import numpy as np
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
from tensorflow.keras.preprocessing import image

# 1. Carrega a MobileNetV2 SEM a última camada (sem cabeça de classificação)
# 'avg' aplica global average pooling, produz vetor (1, 1280)
modelo = MobileNetV2(weights="imagenet", include_top=False, pooling="avg")

def extrair_features_cnn(caminho):
    img = image.load_img(caminho, target_size=(224, 224)) # tamanho ImageNet
    arr = image.img_to_array(img) # (224,224,3)
    arr = preprocess_input(arr[np.newaxis, ...]) # normaliza
    return modelo.predict(arr, verbose=0)[0] # vetor (1280,)

# 2. Aplica em cada maçã do dataset
import glob
X_cnn, y_cnn = [], []
for cls in ["fresh", "rotten"]:
    for p in sorted(glob.glob(f"img/apples/{cls}/*.jpg"))[:80]:
        X_cnn.append(extrair_features_cnn(p))
        y_cnn.append(cls)
X_cnn = np.array(X_cnn) # shape: (160, 1280)
print(X_cnn.shape)
```

A MobileNetV2 carrega **3,5 milhões de parâmetros já treinados**. Não treinamos nada de novo na rede: ela é usada **apenas em modo predição** para gerar vetores.

</> Código: classificar com Regressão Logística

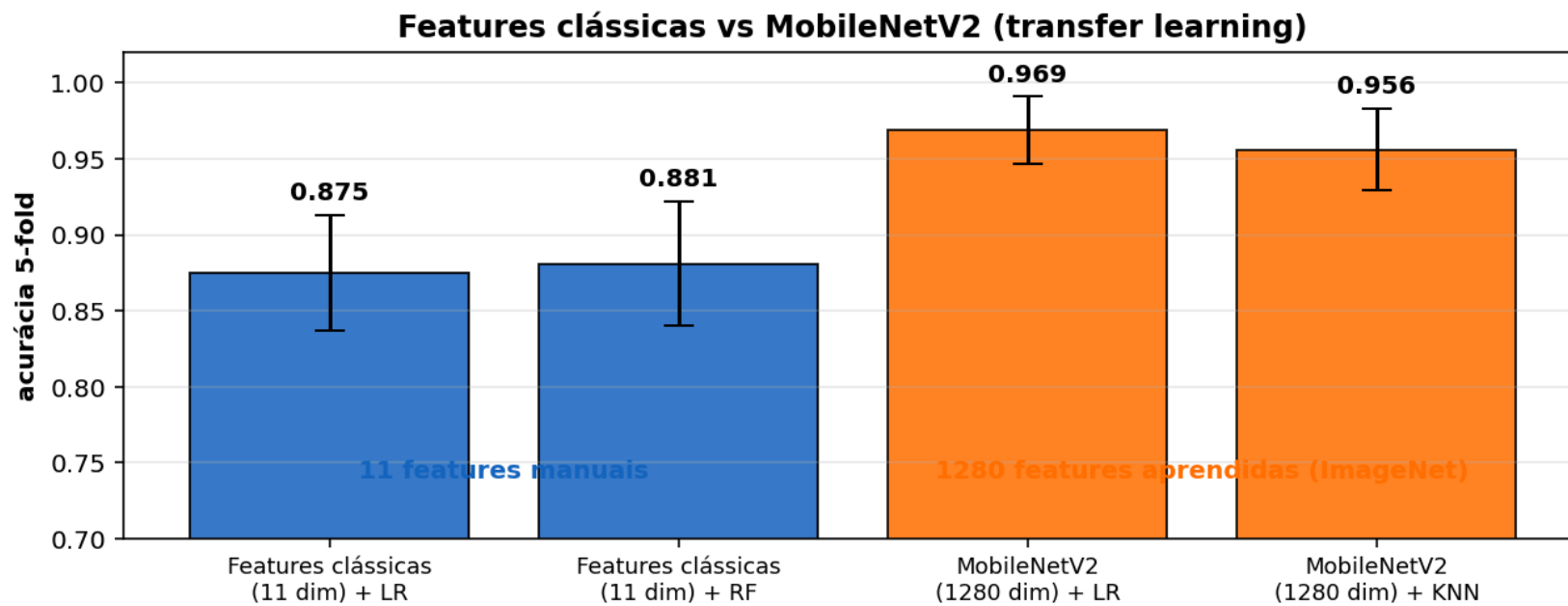
```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score, StratifiedKFold

# Mesma estrutura da seção clássica: scaler + classificador
clf_cnn = Pipeline([
    ("sc", StandardScaler()),
    ("clf", LogisticRegression(C=1.0, max_iter=2000)),
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(clf_cnn, X_cnn, y_cnn, cv=cv, scoring="accuracy")
print(f"Transfer learning (MobileNetV2 + LR): "
      f"{scores.mean():.3f} ± {scores.std():.3f}")
```

Nenhuma linha de código mudou em relação ao pipeline clássico, exceto a fonte do [X](#). É **essa a beleza do transfer learning**: o vetor de 1280 features substitui o vetor de 11 features no mesmo encanamento.

Comparação visual: clássico vs transfer learning



A CNN pré-treinada produz features mais ricas porque foi treinada em **milhão de imagens**. O ganho de acurácia (cerca de 8 a 10 pontos) com **zero treino adicional** explica por que transfer learning virou padrão.

Quando usar cada abordagem

Cenário	Recomendação	Por quê
Poucas imagens (< 200), domínio bem definido	Features clássicas + classificador	Cabe na cabeça do time, debugável, interpretável
Poucas imagens , domínio aberto (várias classes)	Transfer learning + classificador clássico	CNN traz prior visual gigantesco grátis
Milhares de imagens , classes próximas	Fine-tuning da CNN pré-treinada	Refina as últimas camadas para o domínio
Milhões de imagens , recursos GPU	CNN treinada do zero	Tarefa específica vale o custo
Interpretabilidade exigida (auditoria, medicina)	Features clássicas + LR ou árvore	"Por que essa maçã foi classificada como rotten?"

Para o projeto da disciplina

Comece com features clássicas (você já tem o pipeline pronto), reporte a baseline. Depois, suba para transfer learning e mostre o **delta de ganho**. Esse é exatamente o tipo de análise que o seu cliente final quer ver.

Três modos de fazer transfer learning

A literatura distingue **três estratégias** de uso de uma rede pré-treinada, em ordem crescente de complexidade e de necessidade de dados:

Modo	Backbone	Cabeça nova	Quando usar
(1) Feature extractor (foi o que fizemos)	congelado (sem treino)	classificador clássico (LR, SVM, KNN) sobre o vetor de saída	poucos dados (50 a 500/classe), domínio próximo do ImageNet
(2) Fine-tuning parcial	algumas últimas camadas descongeladas	rede Dense pequena treinada do zero	dados médios (500 a 5.000/classe), domínio mais distante
(3) Fine-tuning total	todo o backbone descongelado	rede Dense pequena	muitos dados (> 10.000/classe), recursos de GPU disponíveis

Receita prática para o modo 2 (fine-tuning parcial)

1. Treine só a cabeça nova por algumas épocas, com o backbone **congelado** (modo 1).
2. **Descongele** as últimas N camadas do backbone (ex.: últimos 30 blocos da MobileNetV2).
3. Reduza o *learning rate* para **1e-5** (10 a 100x menor que o usado na etapa 1) para **não destruir** os pesos pré-treinados.
4. Treine mais algumas épocas, monitorando a *loss* de validação.

O passo 1 sozinho já cobre a maioria dos problemas industriais com dados limitados. Só vá para 2 ou 3 se a métrica do passo 1 não bastar.

Catálogo de redes pré-treinadas

Todas as redes abaixo estão prontas em [tensorflow.keras.applications](#) (e em [torchvision.models](#) para PyTorch).

Família	Ano	Parâmetros	Trade-off principal	Caso típico
VGG16 / VGG19	2014	138 M	simples, didática, pesada	baseline pedagógica
ResNet50 / 101 / 152	2015	25 / 44 / 60 M	conexões residuais, robusta, padrão de comparação	servidor com GPU
MobileNetV2 / V3	2018 / 19	3,5 / 5,4 M	leve e rápida , ótima para CPU e mobile	edge, embarcado, demos
EfficientNet-B0 a B7	2019	5,3 a 66 M	melhor relação acurácia/tamanho atual entre as CNNs	quando se quer SOTA sem ViT
ConvNeXt	2022	28 a 200 M	CNN "modernizada" que compete com Transformers	acurácia alta, ainda CNN
Vision Transformer (ViT)	2020	86 a 632 M	atenção em vez de convolução, exige muitos dados	datasets grandes
CLIP / DINOv2	2021 / 23	86 a 1.100 M	features visuais multimodais sem rótulos manuais	zero-shot, similaridade

Os pesos são baixados na primeira execução e ficam em cache local. As dicas práticas de escolha estão no próximo slide.

📌 Como escolher uma rede pré-treinada na prática

Regras de bolso

- **Default sensato:** MobileNetV2 (rápida) ou EfficientNet-B0 (boa relação acurácia/custo).
- **Se sobrar GPU:** ResNet50 segue como padrão de comparação em papers.
- **Domínio muito diferente** de fotos naturais (médico, satélite, microscopia): considere **DINOv2** ou modelos especializados (RadImageNet, BiomedCLIP).
- **Pouca memória** (mobile, edge): MobileNetV3-small.
- **Quer máxima qualidade sem se preocupar com custo:** ConvNeXt ou ViT-Large.

Atalho mental

Modelo	Análogo
ResNet50	sedan confiável
MobileNet	motoneta
EfficientNet	híbrido econômico
ViT	esportivo de pista longa
CLIP / DINOv2	utilitário multifuncional

Não decore o catálogo: aprenda a **olhar a documentação** do **keras.applications** quando precisar.

De onde vêm os pesos: o ImageNet

Quase todas as redes do slide anterior foram pré-treinadas em uma versão do [ImageNet](#):

Os números

- Idealizado por [Fei-Fei Li](#) (Stanford), em 2009.
- [1,4 milhão de imagens](#) rotuladas à mão.
- [1.000 classes](#) (do "papagaio-cinza" ao "termômetro").
- Categorização baseada em [WordNet](#).
- Rotulagem por [Amazon Mechanical Turk](#) (anos de esforço, mais de 49 mil rotuladores).

Por que isso importa

- A competição [ILSVRC](#) (2010 a 2017) virou a "Copa do Mundo" da CV.
- Em 2012, [AlexNet](#) (Krizhevsky, Sutskever, Hinton) venceu com [CNN + GPU](#): marco que iniciou a era do deep learning.
- Em 2015, [ResNet](#) (He et al., Microsoft) ultrapassou a acurácia humana.
- Treinar do zero hoje exige [dias em múltiplas GPUs](#) e energia equivalente a meses de uso doméstico.

Quando você usa [MobileNetV2\(weights="imagenet"\)](#), está reaproveitando [centenas de horas de GPU](#) de um time que treinou aquilo uma vez para a humanidade inteira.

Outros datasets de pré-treino populares: [COCO](#) (detecção), [Places365](#) (cenar), [LAION-5B](#) (5 bilhões de pares imagem-texto, usado pelo CLIP), [JFT-300M / 3B](#) (interno do Google).

</> Exemplo extra: transfer learning multiclasse (não rodado)

Suponha **5 classes de fruta** (maçã, banana, laranja, uva, manga). O código muda muito pouco em relação ao binário: só a cabeça nova e a função de loss.

```
from tensorflow.keras import layers, Model
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

IMG_SIZE, N_CLASSES = 224, 5      # mude aqui: 2 (binario), 5, 100, ...

backbone = MobileNetV2(input_shape=(IMG_SIZE, IMG_SIZE, 3),
                        include_top=False, weights="imagenet")
backbone.trainable = False

inputs = layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = preprocess_input(inputs)
x = backbone(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
x = layers.Dense(64, activation="relu")(x)
outputs = layers.Dense(N_CLASSES, activation="softmax")(x)  # <-- softmax
modelo = Model(inputs, outputs)

modelo.compile(optimizer="adam",
               loss="sparse_categorical_crossentropy",      # <-- multiclasse
               metrics=["accuracy"])
# modelo.fit(ds_train, validation_data=ds_val, epochs=8)
```

3 mudanças em relação ao binário: `N_CLASSES`, ativação `softmax` (em vez de `sigmoid`), e loss `sparse_categorical_crossentropy` (em vez de `binary_crossentropy`). Tudo o mais é idêntico. Para **fine-tuning parcial**: depois faça `backbone.trainable = True`, congele manualmente as primeiras camadas e recompile com `learning_rate=1e-5`.

Se vocês forem projetar uma CNN um dia: dicas práticas

Mesmo fora do escopo desta disciplina, vale levar **intuições de design** caso vocês explorem CNNs em projetos próprios, em uma disciplina futura, ou no mestrado:

Princípios de design

- **Comece pequeno:** 3 a 4 blocos **Conv + Pool**. Adicione profundidade só se a *loss* travar.
- **Cresça canais, encolha espacial:** a cada bloco, **dobre os filtros** (16 → 32 → 64) e **reduza** a resolução (**MaxPool 2x2**).
- **Kernel 3x3 é o padrão:** vários **3x3** empilhados equivalem a um kernel maior, mas com menos parâmetros.
- **GlobalAveragePooling em vez de Flatten:** reduz parâmetros e overfitting na saída.
- **Dropout 0.2 a 0.5** antes da camada Dense final, especialmente com poucos dados.
- **Augmentation** (flip, zoom, rotation) é quase obrigatória com menos de 1000 imagens por classe.

Atalho final: se a sua CNN do zero não bater um pipeline clássico bem feito, **não é arquitetura ruim, é falta de dados**. É exatamente para esse caso que o transfer learning existe.

Sinais para diagnosticar

- **Loss de treino cai, val_loss sobe:** overfitting. Reduza profundidade, aumente dropout, mais augmentation.
- **Ambas travam alto:** underfitting. Adicione capacidade (mais filtros) ou treine mais épocas.
- **Loss oscila violentamente:** **learning_rate** muito alto, divida por 10.
- **Acurácia presa em 50% (binário) ou 1/N (multiclasse):** rede não aprende nada. Verifique normalização e label encoding.
- **val_accuracy > train_accuracy:** provavelmente bug de dataset (vazamento, embaralhamento).

Esses *sintomas* funcionam como um **kit de primeiros socorros** ao treinar qualquer CNN.

RESUMO E PRÓXIMA AULA

👍 O que ficou pronto nesta aula

Seleção de features

- `VarianceThreshold` (filter sem alvo)
- `SelectKBest` com `f_classif` ou `mutual_info_classif`
- `RFE` / `RFECV` (wrapper)
- `feature_importances_` do RF (embedded)
- Combinação: filter → wrapper → embedded

Classificadores clássicos

- KNN (`KNeighborsClassifier`)
- Regressão Logística (`LogisticRegression`)
- SVM (`SVC` com kernels linear ou RBF)
- Random Forest (`RandomForestClassifier`)

■ Pipeline completo da disciplina: `segmentar` (Aulas 6 e 7) → `descrever` (Aula 8) → `selecionar e classificar` (Aula 9).

Avaliação

- `train_test_split` com `stratify`
- `Pipeline` para evitar vazamento de teste
- `cross_val_score` + `StratifiedKFold`
- Matriz de confusão, ROC, AUC

Transição para Deep Learning

- Limites das features manuais
- Neurônio artificial e MLP
- `Transfer learning` com MobileNetV2
- Comparação 11 features vs 1280 features

ENCERRAMENTO DA DISCIPLINA

O caminho completo que vocês percorreram

Aula 1	-> Imagem digital, histogramas, representação
Aula 2	-> Filtragem espacial (médias, gaussiano, mediana)
Aula 3	-> Transformações de intensidade
Aula 4	-> Detecção de bordas (Sobel, Canny)
Aula 5	-> Morfologia matemática
Aula 6	-> Segmentação por limiarização (Otsu)
Aula 7	-> Segmentação avançada (Watershed, GrabCut, SLIC)
Aula 8	-> Extração de features (forma, cor, textura, SIFT)
Aula 9	-> Seleção de features + 4 classificadores + transfer learning

9 aulas, um pipeline completo. Vocês saem capazes de pegar uma imagem qualquer, isolar o objeto, descrever numericamente, escolher as melhores features, treinar um classificador e avaliá-lo de forma rigorosa. Esse é o **kit clássico** que continua sendo usado em produção em muitos setores.

O que vocês conseguem fazer agora

- Implementar um **sistema de inspeção visual automática** do zero
- Decidir **quando** usar features clássicas vs transfer learning vs CNN
- Avaliar um classificador com **rigor estatístico** (CV estratificada, métricas além da acurácia, matriz de confusão)
- Defender suas escolhas tecnicamente em uma entrevista de emprego ou em um TCC

📍 Onde focar o tempo de estudo

Nem todo conteúdo da disciplina tem o mesmo peso para prova, projeto e carreira. Use esta priorização:

Alta prioridade (revisão obrigatória)

1. **segmentação por Otsu + morfologia** (Aulas 6 e 7): aparece em **todo** problema de inspeção, é a base de tudo que vem depois.
2. **regionprops_table + features de forma e cor** (Aula 8): a "linguagem comum" de toda a disciplina; saber quais features extrair e por quê.
3. **Pipeline scikit-learn** (Aula 9): **StandardScaler** → **classificador**, sempre dentro de **Pipeline**, sempre com **train_test_split** estratificado. Falha aqui = nota zero em avaliação.
4. **Matriz de confusão + precisão/recall/F1** (Aula 9): saber **interpretar**, não só calcular. *"Quando posso usar só acurácia? Quando recall importa mais que precisão?"*

Média prioridade

1. **GLCM e LBP** para textura (Aula 8): conceitos quando textura é a feature chave (madeira, tecido, mofo).
2. **Seleção de features: SelectKBest** (filter), **SelectFromModel** (embedded), **RFECV** (wrapper). Saber **escolher** o método conforme custo computacional e tipo de modelo.
3. **Transfer learning** com MobileNetV2: saber a receita de feature extractor (modo 1).

Baixa prioridade (cultura geral)

1. SIFT/ORB e matching: importante para object detection, raro em classificação por região.
2. Detalhes internos de Random Forest e SVM (margem máxima, Gini, etc.): basta saber **quando usar** cada um.
3. Catálogo completo de redes pré-treinadas (ResNet, ViT, etc.): use como referência, não decore.

Regra: se você sabe explicar **por que** uma técnica é usada e **quando ela falha**, está na priorização correta. Decorar fórmula sem entender intuição vale pouco.

⚠ Erros frequentes a evitar (no projeto e na carreira)

Erros metodológicos que zeram a credibilidade

- **Vazamento de informação**: ajustar `StandardScaler`, `PCA` ou seleção de features **vendo o conjunto de teste**. Sempre dentro de `Pipeline` + `cross_val_score`.
- **Avaliar só com acurácia** em dataset desbalanceado: 95% de acurácia pode ser "predizer sempre a classe majoritária".
- **Esquecer `stratify=y`** em `train_test_split`: gera teste com proporção de classes errada.
- **Não fixar `random_state`**: resultado muda a cada execução, ninguém consegue reproduzir.

Erros conceituais comuns

- Confundir **extração** e **seleção** de features.
- Achar que **mais features sempre é melhor**: features redundantes prejudicam KNN, SVM e Regressão Logística.
- Pular **boxplot por classe** antes de treinar: você deveria saber se as features separam **visualmente** antes de gastar tempo em modelo.
- Tratar **CNN/transfer learning como mágica**: sem entender o pipeline clássico, você não consegue julgar se a CNN realmente está ganhando ou só decorando o treino.

Para aprofundar depois desta disciplina

Caminhos sugeridos por [interesse](#), em ordem crescente de profundidade:

Curto prazo (semanas)

- Refazer o [projeto AI com um dataset próprio](#) (frutas do mercado, peças do seu trabalho, fotos do celular)
- Praticar no [Kaggle](#): começar pelos *Getting Started Competitions* (Digit Recognizer, Titanic)
- Acompanhar [fast.ai](#) (curso gratuito, prático, em vídeo)

Médio prazo (meses)

- Estudar [CNN do zero](#) em [CS231n de Stanford](#) (vídeos no YouTube, materiais abertos)
- Implementar [Grad-CAM](#) sobre uma CNN pré-treinada
- Construir um [deploy](#) com Streamlit + Docker (o demo de maçãs já te dá a base)

Visão computacional é uma área que muda rápido, mas o [pipeline clássico que vocês aprenderam aqui não envelhece](#). Toda solução moderna ainda passa por pré-processamento, escolha de representação e avaliação rigorosa. Vocês têm o alicerce.

Longo prazo (anos)

- Especializações modernas em [Vision Transformers \(ViT\)](#), [diffusion models](#), [multimodal \(CLIP, LLaVA\)](#)
- Participar de [competições reais](#) (Kaggle, AI Singapore, DrivenData)

Livros e referências de cabeceira

- [Gonzalez & Woods](#), *Digital Image Processing* (clássico de CV)
- [Goodfellow, Bengio, Courville](#), *Deep Learning* (gratuito em [deeplearningbook.org](#))
- [Murphy](#), *Probabilistic Machine Learning* (formal, completo)
- [Géron](#), *Hands-On ML with Scikit-Learn, Keras and TensorFlow* (prático)

OBRIGADO!

Sucesso no projeto e nas próximas disciplinas.

Prof. Dr. José Jairo de S. e Silva

jjairo@up.edu.br

Referências

Bibliografia principal

- GONZALEZ, R. C.; WOODS, R. E. *Digital Image Processing*. 4th ed. Pearson, 2018. [Cap. 12: Image Pattern Classification](#).
- HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. *The Elements of Statistical Learning*. 2nd ed. Springer, 2009. (caps. 4, 7, 9, 15)
- GUYON, I.; ELISSEEFF, A. *An Introduction to Variable and Feature Selection*. JMLR, 2003.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. MIT Press, 2016. (caps. 5, 6, 9)
- PAN, S. J.; YANG, Q. *A Survey on Transfer Learning*. IEEE TKDE, 2010.
- YOSINSKI, J. et al. *How transferable are features in deep neural networks?* NeurIPS, 2014.

Documentação consultada

- scikit-learn: [feature_selection](#), [pipeline](#), [model_selection](#), [metrics](#)
- scikit-learn: [KNeighborsClassifier](#), [LogisticRegression](#), [SVC](#), [RandomForestClassifier](#)
- TensorFlow / Keras: [MobileNetV2](#), [Transfer learning tutorial](#)

EXERCÍCIOS EM PYTHON

 Exercício 1: ranking de features (filter)

Use as 11 features clássicas e descubra **quais separam mais** as classes **fresh** e **rotten**. Compare ANOVA e mutual information.

```
import pandas as pd, numpy as np
from sklearn.feature_selection import (
    f_classif, mutual_info_classif, SelectKBest)

# X, y vêm do recap: 160 maçãs x 11 features
F, p = f_classif(X, y)
mi = mutual_info_classif(X, y, random_state=0)

ranking = pd.DataFrame({
    "feature": nomes,
    "F": F.round(2),
    "p": p,
    "MI": mi.round(3),
}).sort_values("F", ascending=False)
print(ranking.to_string(index=False))

top5_f = SelectKBest(f_classif, k=5).fit(X, y)
top5_mi = SelectKBest(mutual_info_classif, k=5).fit(X, y)
print(set(np.array(nomes)[top5_f.get_support()])
      ^ set(np.array(nomes)[top5_mi.get_support()])))
```

Perguntas

- Quais são as 3 features com maior **F**? Pertencem a qual família: **cor**, **forma** ou **textura**?
- Quais features têm **p > 0,05**? Elas dificilmente carregam sinal e podem ser **descartadas** sem perda.
- O conjunto **top-5 por F** coincide com o **top-5 por MI**? A diferença simétrica Δ te diz quais features só aparecem em um dos rankings, sugerindo relação **não linear** com a classe.
- Desafio:** recompute o **f_classif** removendo a feature **S_mean** (que costuma dominar). O ranking das demais muda muito? Isso indica multicolinearidade.

 Exercício 2: RFECV para escolher k automaticamente

RFECV faz wrapper com validação cruzada. Use-o para descobrir o **número ótimo** de features para uma SVM linear.

```
import matplotlib.pyplot as plt
from sklearn.feature_selection import RFECV
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold

X_norm = StandardScaler().fit_transform(X)

rfe = RFECV(
    estimator=SVC(kernel="linear", C=1.0),
    step=1,
    cv=StratifiedKFold(5, shuffle=True, random_state=0),
    scoring="accuracy",
    min_features_to_select=1,
)
rfe.fit(X_norm, y)

ks = range(1, len(rfe.cv_results_["mean_test_score"]) + 1)
plt.plot(ks, rfe.cv_results_["mean_test_score"], "o-",
         color="#1565c0")
plt.xlabel("nº de features"); plt.ylabel("acurácia 5-fold")
plt.title("RFECV"); plt.grid(alpha=0.3); plt.show()

print(f"k ótimo: {rfe.n_features_}")
print("Mantidas:", [n for n, m in zip(nomes, rfe.support_) if m])
```

Perguntas

- Para quantas features o **RFECV** chegou? A curva é **monotônica** ou tem um pico claro?
- Compare o conjunto escolhido com o **top-5** do Exercício 1. Coincide muito?
- Repita o **RFECV** trocando **SVC(kernel="linear")** por **LogisticRegression()**. As features escolhidas mudam? Por quê?
- Desafio:** rode **RFECV** com **RandomForestClassifier(n_estimators=200)**. Compare o **k_ótimo** e as features mantidas com as duas variantes lineares. RFs costumam preferir conjuntos **maiores** porque consomem qualquer feature útil sem prejuízo.

 Exercício 3: comparar 4 classificadores

Treine KNN, Regressão Logística, SVM e Random Forest no mesmo X , y . Reporte média e desvio padrão da acurácia em 5-fold CV.

```
import pandas as pd
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

clfs = {
    "KNN": Pipeline([("sc", StandardScaler()),
                     ("clf", KNeighborsClassifier(5, weights="distance"))]),
    "LogReg": Pipeline([("sc", StandardScaler()),
                       ("clf", LogisticRegression(max_iter=1000))]),
    "SVM RBF": Pipeline([("sc", StandardScaler()),
                        ("clf", SVC(kernel="rbf", probability=True))]),
    "RF": RandomForestClassifier(n_estimators=300, random_state=0),
}

cv = StratifiedKFold(5, shuffle=True, random_state=42)

linhas = []
for nome, clf in clfs.items():
    s = cross_val_score(clf, X, y, cv=cv, scoring="accuracy")
    linhas.append({"clf": nome,
                  "media": s.mean(), "desvio": s.std()})
print(pd.DataFrame(linhas).round(3).to_string(index=False))
```

Perguntas

- Qual classificador teve a **maior média**? E o **menor desvio padrão**? (Eles costumam ser classificadores diferentes.)
- Repita usando só as 5 features escolhidas no Exercício 2. A diferença é grande? Em qual classificador você esperaria **mais ganho** ao remover ruído?
- Troque a métrica para **scoring="f1"** (mais sensível a desbalanceamento). O ranking muda?
- Desafio:** rode **GridSearchCV** para tunar **n_neighbors** no KNN ({3,5,7,11,15}) e **C** no SVM ({0.1,1,10}). O melhor **n_neighbors** é o mesmo dos vizinhos clássicos da literatura (5 ou 7)?

 Exercício 4: matriz de confusão e ROC

Treine **Regressão Logística** com `train_test_split` (75/25, estratificado). Mostre a matriz de confusão e a curva ROC.

```
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    ConfusionMatrixDisplay, RocCurveDisplay,
    classification_report, roc_auc_score)

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

lr = Pipeline([("sc", StandardScaler()),
               ("clf", LogisticRegression(max_iter=1000))])
lr.fit(X_tr, y_tr)
y_pred = lr.predict(X_te)
y_prob = lr.predict_proba(X_te)[:, 1]

print(classification_report(y_te, y_pred,
                           target_names=["fresh", "rotten"]))
print("AUC:", roc_auc_score(y_te, y_prob))

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ConfusionMatrixDisplay.from_predictions(
    y_te, y_pred, display_labels=["fresh", "rotten"],
    ax=ax[0], cmap="Blues")
RocCurveDisplay.from_predictions(y_te, y_prob, ax=ax[1])
plt.tight_layout(); plt.show()
```

Perguntas

- Onde estão os erros: o classificador confunde mais **fresh em rotten** (FP), ou **rotten em fresh** (FN)? Explique o que isso significa no contexto de uma linha de inspeção.
- Suba o limiar para `0.7` (`(y_prob > 0.7).astype(int)`) e refaça a matriz. O que aumenta: precisão ou recall?
- Imprima os 5 coeficientes de maior valor absoluto da Regressão Logística (`lr.named_steps["clf"].coef_[0]`). O sinal bate com a intuição (frac_brown alto deve empurrar para **rotten**)?
- Desafio:** plote a curva **precision-recall** no lugar da ROC (`PrecisionRecallDisplay.from_predictions`). Em datasets desbalanceados, ela é mais informativa que a ROC.

 Exercício 5: transfer learning com MobileNetV2

Extraia features das 160 maçãs com a MobileNetV2 pré-treinada e treine uma Regressão Logística. Compare com o resultado do Exercício 3.

```
import glob, numpy as np
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
from tensorflow.keras.preprocessing import image
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score, StratifiedKFold

modelo = MobileNetV2(weights="imagenet",
                      include_top=False, pooling="avg")

def feat(p):
    img = image.load_img(p, target_size=(224, 224))
    arr = image.img_to_array(img)
    arr = preprocess_input(arr[None, ...])
    return modelo.predict(arr, verbose=0)[0]

X_cnn, y_cnn = [], []
for cls in ["fresh", "rotten"]:
    for p in sorted(glob.glob(f"img/apples/{cls}/*.jpg"))[:80]:
        X_cnn.append(feat(p)); y_cnn.append(cls)
X_cnn = np.array(X_cnn)

clf = Pipeline([("sc", StandardScaler()),
               ("clf", LogisticRegression(max_iter=2000))])
s = cross_val_score(clf, X_cnn, y_cnn,
                    cv=StratifiedKFold(5, shuffle=True, random_state=42),
                    scoring="accuracy")
print(f"TL + LR: {s.mean():.3f} ± {s.std():.3f}")
```

Perguntas

- Compare a acurácia obtida com a **melhor** do Exercício 3 (features clássicas). Qual é a diferença em pontos percentuais?
- `X_cnn.shape` é **(160, 1280)**. O classificador tem **muito mais features que amostras**. Por que ele ainda funciona? (Dica: pense em regularização L2 da Regressão Logística.)
- Aplique **PCA(n_components=2)** em `X_cnn` e plote um scatter colorido por classe. As classes ficam **visivelmente separadas**? Faça o mesmo com `X` clássico (11 dim) e compare.
- Desafio:** troque a MobileNetV2 por **ResNet50** (também em `keras.applications`). O ganho compensa o aumento de tamanho do modelo (25M vs 3,5M parâmetros)?

 Exercício 6: pipeline integrado

Monte um **único pipeline** que: padroniza, seleciona features e classifica. Avalie com 5-fold CV.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, StratifiedKFold

pipe = Pipeline([
    ("sc", StandardScaler()),
    ("sel", SelectKBest(score_func=f_classif)),
    ("clf", LogisticRegression(max_iter=1000)),
])

grade = {
    "sel__k": [3, 5, 7, 11],          # quantas features manter
    "clf__C": [0.1, 1.0, 10.0],      # regularização
}

busca = GridSearchCV(
    pipe, grade,
    cv=StratifiedKFold(5, shuffle=True, random_state=42),
    scoring="accuracy", n_jobs=-1,
)

busca.fit(X, y)
print("Melhor combinação:", busca.best_params_)
print(f"Acurácia: {busca.best_score_:.3f}")
print("Features escolhidas:",
      [n for n, m in zip(nomes,
                        busca.best_estimator_.named_steps['sel'].get_support()) if m])
```

Perguntas

- Qual **k** venceu? Qual **C**? A combinação faz sentido (poucos dados pedem mais regularização e menos features)?
- Por que essa estrutura evita **vazamento de informação**? (Pense: a seleção de features acontece **dentro** de cada fold do CV.)
- Adicione **SVC(kernel="rbf")** no lugar da LR, com **clf__gamma** na grade. O melhor **k** muda?
- Desafio:** substitua **SelectKBest** por **PCA(n_components=k)**. Qual produz melhor acurácia: **selecionar k features originais** ou **projetar em k componentes**? O PCA perde interpretabilidade, vale a troca?